

Graph-Based Machine Learning for Detecting Lateral Movement in Enterprise Networks

ENPM604 Research Paper

Suthirtha Nagesh

May 2026

One-line summary: This paper explores how graph-based machine learning can model relationships between users, hosts, processes, and network connections to detect lateral movement attacks that traditional rule-based security tools may miss.

Abstract

Lateral movement is one of the harder stages of an enterprise compromise to detect. After attackers gain an initial foothold, they often use valid accounts, remote services, and normal administrative paths to move from one internal system to another. A single login or network connection may not look malicious by itself, but the full chain of activity can reveal a suspicious path. This paper studies graph-based machine learning as a practical way to model those relationships. The main idea is to build an enterprise activity graph from authentication, process, DNS, and network-flow logs, learn normal relationship patterns, and flag low-probability edges or paths as possible lateral movement. The paper focuses on graph embeddings, link prediction, graph neural networks, and temporal graph learning. Based on the literature, graph-based methods are promising because they preserve context that isolated log rules often lose. At the same time, real deployment would need to handle false positives, model drift, data quality issues, explainability, and large-scale graph processing.

1 Introduction

Enterprise attacks usually do not stop at the first compromised machine. After getting in, an attacker may try to discover internal systems, collect credentials, pivot through hosts, and eventually reach high-value resources such as file servers, domain controllers, databases,

or cloud administration consoles. MITRE ATT&CK describes lateral movement as the techniques adversaries use to enter and control remote systems inside a network. In many cases, the attacker is not using obviously malicious traffic. They may be using valid credentials, remote access tools, and internal services that already exist in the environment (MITRE ATT&CK, n.d.-a).

This makes lateral movement a difficult detection problem. If a defender only looks at one log entry at a time, many events can appear normal. A user logging into a host, a process starting, or a server connection may all be legitimate in isolation. The suspicious behavior often appears only when those events are connected as a path. For that reason, this paper treats lateral movement as a relationship problem rather than only a packet, process, or log-line classification problem.

The paper studies how graph-based machine learning can help with this problem. In a graph, users, hosts, processes, services, and IP addresses can be represented as nodes, while logins, process launches, DNS lookups, and network connections can be represented as edges. A graph model can then learn which relationships are normal for the organization and which relationships or paths are unusual. Instead of only asking, “Does this event match a known rule?”, the model can ask, “Is this edge or path unlikely based on past enterprise behavior?”

The proposed solution is not meant to be a fully autonomous blocking system. A more realistic goal is to use graph-based ML as an alert-prioritization layer that supports a security operations center. The model would generate an anomaly score and provide context such as the user, source host, destination host, process, time window, and suspicious path. This gives analysts more useful information than a generic alert that only says “rare login” or “unusual connection.”

2 Research Problem and Purpose

The specific problem addressed in this research is that traditional rule-based tools can miss lateral movement when attackers blend into normal enterprise behavior. Static rules are useful for known malware, suspicious commands, or clear policy violations, but lateral movement is often more subtle. For example, a domain user may authenticate to a workstation, use a remote administration tool, and then connect to an internal server. Each event may have a reasonable explanation by itself, but the complete path may be unusual for that user, role, host, or time of day.

The purpose of this research is to evaluate whether graph-based machine learning is a good fit for detecting this type of internal movement. The central question is: can graph-based ML learn normal relationships among accounts, hosts, processes, and network connections, then flag low-probability movement paths that may indicate lateral movement? This is a cybersecurity problem because early detection of lateral movement can help defenders respond

before an attacker reaches sensitive systems or begins data theft.

A second purpose is to keep the approach practical. In a real SOC, a detection system must be useful to analysts, not just accurate on paper. A model that produces thousands of unexplained alerts per day would create more noise than value. For this reason, this paper emphasizes low false-positive rates, explainable paths, and integration with existing SIEM and EDR workflows.

3 Background: Why Graphs Fit Lateral Movement

A graph represents objects as nodes and relationships as edges. For this topic, nodes can represent users, hosts, IP addresses, processes, services, files, or domain accounts. Edges can represent authentications, remote logins, process launches, DNS lookups, network connections, or file access. This structure fits lateral movement because an intrusion is usually not one isolated event. It is a chain of actions across accounts, machines, and services.

Public cybersecurity datasets also show why this representation is realistic. The Los Alamos National Laboratory Comprehensive Multi-Source Cyber-Security Events dataset contains 58 days of de-identified enterprise data from multiple sources, including Windows authentication events, Active Directory domain controller events, process start and stop events, DNS lookups, network-flow data, and red-team events (Kent, 2015; Los Alamos National Laboratory, n.d.). These are the kinds of logs that can be converted into a heterogeneous enterprise graph.

Table 1 shows a simplified example of how raw enterprise activity can be represented as graph data.

Table 1. Example graph representation of enterprise security data

Entity or event	Graph representation	Example feature	Detection value
User account	Node	Role, department, normal hosts	Identify compromised or abused credentials
Host or server	Node	Asset criticality, subnet, server type	Find unusual access to sensitive systems
Login event	Edge	Source, destination, time, success/failure	Detect rare user-host relationships
Process execution	Edge or node	Process name, parent process, command type	Add context to remote activity

Entity or event	Graph representation	Example feature	Detection value
Network connection	Edge	Port, protocol, frequency, direction	Detect pivoting and unexpected service access

Graph modeling also supports both edge-level and path-level reasoning. Edge-level detection asks whether a specific login, connection, or user-host pair is unlikely. Path-level detection asks whether a sequence of events resembles movement from one compromised asset to another. This is closer to how a SOC analyst investigates an incident, because the analyst usually needs to understand the chain of activity, not just one alert.

4 Proposed Approach

The proposed approach is a research framework, not a claim that one model will solve every lateral movement case. The framework has five stages: data collection, graph construction, feature engineering, graph-based learning, and analyst-facing alerting.

- **Data collection:** Collect authentication logs, endpoint or process logs, DNS logs, network-flow logs, and SIEM/EDR alerts for a selected time period.
- **Graph construction:** Create nodes for users, hosts, processes, services, and IP addresses. Create typed edges for logins, process launches, connections, and lookups.
- **Feature engineering:** Add features such as time of day, edge frequency, role context, asset criticality, protocol, and whether the edge is new or rare for that user or host.
- **Graph-based learning:** Learn normal enterprise relationships using graph embeddings, link prediction, graph neural networks, or temporal graph models.
- **Alerting:** Generate anomaly scores and explain suspicious paths. For example, an alert may state that User U42 rarely logs into Host H17 and then reaches FinanceSrv through remote administration tooling.

A key design choice is to begin with unsupervised or semi-supervised learning. In real enterprises, labeled lateral movement examples are rare. Organizations may not know exactly when an attacker was present, and attack labels may be incomplete. Unsupervised graph learning is attractive because it can learn the normal structure of the environment and flag deviations without needing a large labeled attack dataset.

As a possible small prototype, the approach could be tested on a subset of the LANL authentication and red-team data. Authentication events can be converted into user-host and host-host edges. A GraphSAGE link-prediction model could then learn likely normal edges and score low-probability edges as suspicious candidates. This would not be a full production detector, but it would be enough to demonstrate how real enterprise logs can be transformed into a graph-based ML problem.

5 Algorithms and Techniques Featured

This research focuses on three levels of methods: simple baselines, graph embedding with link prediction, and graph neural networks or temporal graph models.

The first level is a baseline, such as rule-based or statistical rare-event detection. For example, a simple detector may flag a user logging into a host they have never accessed before. This baseline matters because it gives a fair comparison point. If a complex graph model does not perform better than a simple rare-login rule, then the added complexity may not be worth it. The weakness of this baseline is that rare events are not always malicious. New projects, helpdesk work, patching, vulnerability scanning, and administrator troubleshooting can all create rare edges.

The second level is graph embedding with link prediction. A model can learn vector representations of users and hosts based on historical graph structure. A link predictor then estimates how likely a user-host or host-host relationship is. If the predicted probability is very low, the edge can be treated as anomalous. Bowman et al. (2020) used unsupervised graph learning and a link-prediction-style approach to detect lateral movement in enterprise networks. Their work is useful for this topic because it shows that graph structure can improve detection compared with traditional heuristics.

The third level is graph neural networks, especially temporal graph models. EULER frames lateral movement detection as a temporal graph link prediction problem and uses a graph neural network with a sequence-encoding layer to model how enterprise relationships evolve over time (King & Huang, 2023). This is important because enterprise behavior is not fixed. A relationship that is unusual this month may become normal later if an employee changes roles or a new application is deployed. Temporal modeling can help reduce false alerts caused by normal business change.

A practical future extension is explainability. A graph-based alert should explain which edge, path, user, host, or feature contributed most to the anomaly score. This matters because security analysts need to understand and trust the alert before they can investigate it. Even a high-performing model may fail operationally if analysts cannot tell why the alert was generated.

6 Expected Effectiveness and Key Findings

This paper does not claim new experimental results. Instead, it evaluates the proposed approach through findings from current literature. Bowman et al. (2020) reported that their unsupervised graph AI approach detected malicious authentication events associated with lateral movement at an 85% true positive rate and 0.9% false positive rate on real enterprise data. They also reported that traditional heuristics and non-graph anomaly methods produced lower true positive rates and higher false positive rates. This supports the idea that graph structure adds useful detection context.

EULER provides another strong argument for this direction. It formulates lateral movement detection as anomalous link prediction over evolving graphs. This is valuable because enterprise networks change over time as users, machines, and services change (King & Huang, 2023). More recent work such as CyberGFM also explores graph-based foundation models for lateral movement detection, which suggests that this remains an active research area rather than a solved problem (King et al., 2026).

A systematic survey of lateral movement detection grouped the field into endpoint detection and response, machine learning solutions, and graph-based strategies (Smiliotopoulos et al., 2024). Based on these findings, the expected effectiveness of this framework is strongest when graph ML is used to prioritize suspicious paths for analyst review, not when it is used as a fully autonomous decision-maker.

Table 2. Main research evidence used in this paper

Reference	Technique / focus	Useful finding	Relevance to this paper
Bowman et al. (2020)	Unsupervised Graph AI / link prediction	Reported 85% TPR and 0.9% FPR on enterprise data	Supports graph modeling for rare lateral movement edges
King & Huang (2023)	Temporal graph link prediction	Models evolving network interactions over time	Supports temporal graph methods for changing enterprises
LANL dataset	Enterprise logs and red-team events	Provides authentication, process, DNS, flow, and red-team data	Shows the type of evidence needed for graph construction

Reference	Technique / focus	Useful finding	Relevance to this paper
Smiliotopoulos et al. (2024)	Systematic survey	Groups lateral movement work into EDR, ML, and graph strategies	Shows that the topic is current and research-relevant

7 Shortfalls and Limitations

The first shortfall is false positives. In a large organization, rare behavior happens every day. Administrators, helpdesk staff, vulnerability scanners, deployment tools, and emergency troubleshooting can all create unusual edges. A graph model may correctly identify a rare relationship but still be wrong about whether it is malicious. This is why the model needs role context, asset context, allowlists, and analyst feedback.

The second shortfall is model drift. Enterprise networks change constantly. Employees move teams, systems are retired, cloud workloads are added, and new business applications create new traffic patterns. If the model is not updated, normal changes may start to look like attacks. Temporal graph models can help with this issue, but they also make the system more complex.

The third shortfall is training data quality. If the training period contains hidden attacker behavior, the model may learn part of the attack as normal. Logs can also be incomplete, inconsistent, or collected for operations rather than security analytics. The LANL dataset is useful for research, but real enterprise deployments must handle missing fields, sensor gaps, and privacy restrictions.

The fourth shortfall is explainability. A graph neural network can be powerful, but if it only returns a score without a clear explanation, analysts may ignore it. A useful security tool should show why an alert matters: which user, host, edge, time window, or path was abnormal.

The final shortfall is scalability. Enterprise graphs can contain millions or billions of events. Training and querying large graph models can be expensive. A practical design would need time-windowed graphs, sampling, efficient storage, and possibly streaming updates.

8 Mitigation Strategies and Practical Deployment

The main mitigation strategy is to deploy graph-based ML as an assistive detection layer, not as the only security control. Alerts should be integrated into SIEM or EDR workflows

and ranked by risk. Each alert should include context such as the suspicious path, affected accounts, asset criticality, command or process evidence, and related ATT&CK technique. This reduces the chance that analysts treat the model as a black box.

False positives can be reduced by combining graph scores with operational context. Known administrator jump boxes, vulnerability scanners, patch-management servers, and helpdesk workflows can be allowlisted or assigned different thresholds. Analyst feedback can also be used to suppress repeated benign patterns and improve the model over time. The system should use time-aware baselines so that a new relationship does not remain suspicious forever if it becomes part of normal business operations.

Detection should also be paired with preventive controls. MITRE lists network segmentation as a mitigation that can restrict lateral movement by limiting traffic between systems and protecting critical assets (MITRE ATT&CK, n.d.-b). Least privilege, multi-factor authentication, privileged access management, credential rotation, and endpoint hardening reduce how far an attacker can move. Graph ML helps detect suspicious movement, but these controls reduce the attacker’s opportunity to move in the first place.

Privacy and governance also matter. Graph-based analysis may include user behavior, host access patterns, and process activity. Organizations should restrict access to this data, define retention periods, and make sure monitoring is used for security purposes with proper oversight.

9 Conclusion

This paper concludes that graph-based machine learning is well-suited for lateral movement detection because lateral movement is relational. It is not just one suspicious login or one unusual packet. It is a chain of users, hosts, processes, and network connections that can form an unusual path toward sensitive systems. Graph-based models preserve this relationship context better than isolated rule-based detections.

The best solution is probably not one single algorithm. A practical approach should start with simple baselines, then compare graph embeddings, link prediction, and temporal graph neural networks. Prior work such as Bowman et al. (2020) and EULER shows that graph-based methods can improve detection quality, but the system still needs to be designed around low false positives, scalability, and explainability.

Overall, graph-based ML should be used as an analyst-support tool for enterprise detection and response. Its value is highest when it identifies suspicious paths, explains why they are unusual, and helps analysts prioritize investigations. Future work should compare simpler graph embedding models with temporal GNNs on LANL-style datasets, measure both detection metrics and alert volume, and develop explainable alert formats that SOC analysts can actually use during incident response.

References

- Bowman, B., Laprade, C., Ji, Y., & Huang, H. H. (2020). Detecting lateral movement in enterprise computer networks with unsupervised graph AI. RAID 2020.
url<https://www.usenix.org/conference/raid2020/presentation/bowman>
- Kent, A. D. (2015). Comprehensive, Multi-Source Cyber-Security Events Data Set. Los Alamos National Laboratory.
url<https://doi.org/10.17021/1179829>
- King, I. J., & Huang, H. H. (2023). Euler: Detecting network lateral movement via scalable temporal link prediction. NDSS / ACM.
url<https://www.ndss-symposium.org/ndss-paper/auto-draft-227/>
- King, I. J., Trindade, B., Bowman, B., & Huang, H. H. (2026). CyberGFM: Graph foundation models for lateral movement detection in enterprise networks. arXiv.
url<https://arxiv.org/abs/2601.05988>
- Los Alamos National Laboratory. (n.d.). Comprehensive, Multi-Source Cyber-Security Events.
url<https://csr.lanl.gov/data/cyber1/>
- MITRE ATT&CK. (n.d.-a). Lateral Movement, Tactic TA0008.
url<https://attack.mitre.org/tactics/TA0008/>
- MITRE ATT&CK. (n.d.-b). Enterprise Mitigations.
url<https://attack.mitre.org/mitigations/enterprise/>
- Smiliotopoulos, C., Kambourakis, G., & Kolias, C. (2024). Detecting lateral movement: A systematic survey. Heliyon.
url<https://www.sciencedirect.com/science/article/pii/S240584402402348X>